NUS National University of Singapore | School of Computing

CS5340
Uncertainty Modeling in AI

Lecture 11:
Variational Autoencoders and Diffusion Models

Assoc. Prof. Lee Gim Hee

AY 2026/27

Semester 1

Course Schedule

Week	Date	Topic	Remarks
1	10 Aug	National Day Holiday	No Lecture
2	17 Aug	Introduction to probabilistic reasoning	**Assignment 0:** Python Numpy Tutorial (Ungraded)
	22 Aug	Bayesian networks (Directed graphical models)	**Make-up Lecture:** Sat, 9.30am-12.30pm (LT15, Recording will be provided)
3	24 Aug	Markov random Fields (Undirected graphical models)	
4	31 Aug	Variable elimination and belief propagation	**Assignment 1:** Belief propagation and maximal probability (15%) **Assignment 1:** Due (13 Sep, 2359 HRS)
	05 Sept	Factor graph and the junction tree algorithm	**Make-up Lecture:** Sat, 9.30am-12.30pm (LT15, Recording will be provided)
5	07 Sep	I will be traveling	No Lecture
6	14 Sep	Parameter learning with complete data	**Assignment 2:** Junction tree and parameter learning (15%) **Assignment 2:** Due (27 Sep, 2359 HRS)
-	21 Sep	Recess week	No Lecture
7	28 Sep	Mixture models and the EM algorithm	
8	05 Oct	Hidden Markov Models (HMM)	**Assignment 3:** Hidden Markov model (15%) **Assignment 3:** Due (18 Oct, 2359 HRS)
9	12 Oct	Monte Carlo inference (Sampling)	
10	19 Oct	Variational inference	**Assignment 4:** MCMC Sampling (15%) **Assignment 4:** Due (01 Nov, 2359 HRS)
11	26 Oct	Variational Autoencoder and Diffusion Models	
12	02 Nov	Graph-cut and alpha expansion	
13	09 Nov	Diwali/Deepavali	No Lecture

Final Exam: 21 Nov 2026

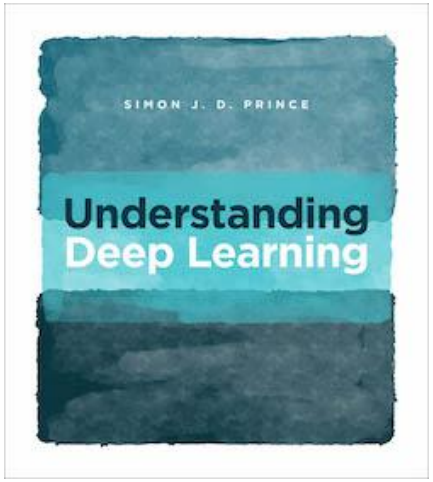
Acknowledgements

- A lot of slides and content of this lecture are adopted from:

1. Carl Doersch, “Tutorial on Variational Autoencoders”, in ArXiv, 2016.

2. Diederik P Kingma, Max Welling, “Auto-Encoding Variational Bayes”, in ICLR 2014.

3. Understanding Deep Learning, Simon Prince MIT Press Dec 5th, 2023.
   Chapters 18



Learning Outcomes

- Students should be able to:

1. Explain the difference between the discriminative and generative models.
2. Describe the concept behind Variational AutoEncoder, and how it can be used to generate new images.
3. Explain the forward (diffusion) and reverse process to noise and denoise an input in the diffusion model.

Recall: Discriminative Vs Generative Models

- Generative models: Approaches that explicitly or implicitly model the distribution of inputs and outputs.

- Sampling from the distribution it is possible to generate synthetic data points in the input space.

Likelihood: $p(\mathbf{x}|\mathcal{C}_k)$

- Discriminative models: Approaches that model the posterior probabilities directly.

Posterior: $p(\mathcal{C}_k|\mathbf{x})$

Discriminative Models

During learning, we model the posterior probability:



$$p(Z = \text{Bedroom} \mid X = \text{  }) = 0.01$$

$$\vdots$$

$$p(Z = \text{Bedroom} \mid X = \text{  }) = 0.90$$

Example: Logistic regression, convolutional network, etc.

Discriminative Models

During inference: Given  , find $p(Z \mid X =$  $)$.

$p(Z = \text{Bedroom} \mid X =$  $)$



Z = Bedroom or Dinning room?

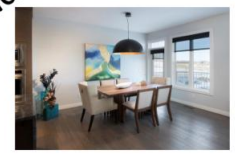


$Z = \text{Bedroom}$





Decision boundary



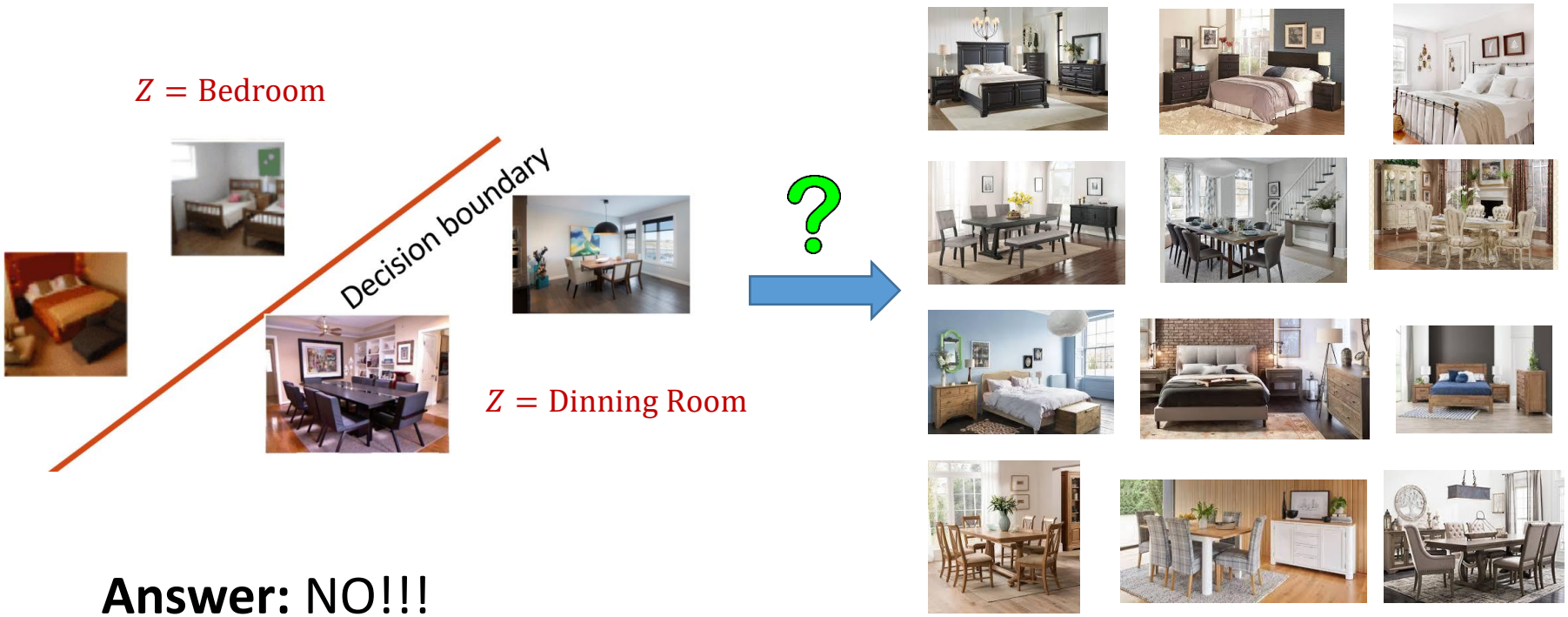


$Z = \text{Dinning Room}$

$p(Z = \text{Dinning Room} \mid X =$  $)$

Discriminative Models

Question: Can we generate samples of new images from the posterior $p(Z \mid X)$?



Answer: NO!!!

Use a generative model instead.

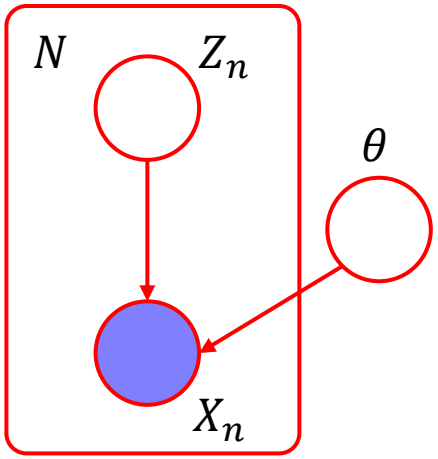
Generative Models

Question: Can we generate samples of new images?

Answer: Yes, use the likelihood model!

Consider the following graphical model, where the joint distribution is given by:

$$p(X, Z \mid \theta) = \underbrace{p(X \mid Z, \theta)}_{\text{likelihood}} \underbrace{p(Z)}_{\text{prior}}$$



Z: latent variable

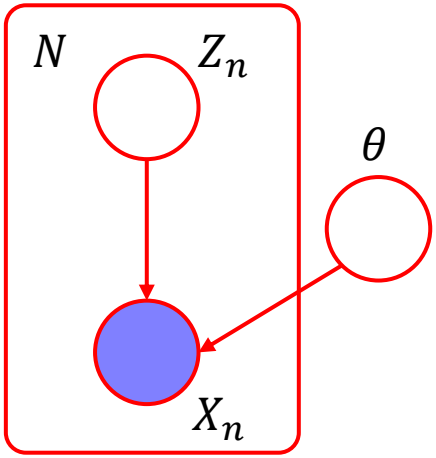
X: observed variable

θ: likelihood model parameter

Generative Models

Consider the following graphical model, where the joint distribution is given by:

$$p(X, Z \mid \theta) = p(\underbrace{X \mid Z, \theta}_{\text{likelihood}})\underbrace{p(Z)}_{\text{prior}}$$



A simple idea:

1. Sample from the prior $z \sim p(Z)$, e.g., $z = \text{Bedroom}$
2. Generate new image from the likelihood $p(X \mid Z = \text{Bedroom})$

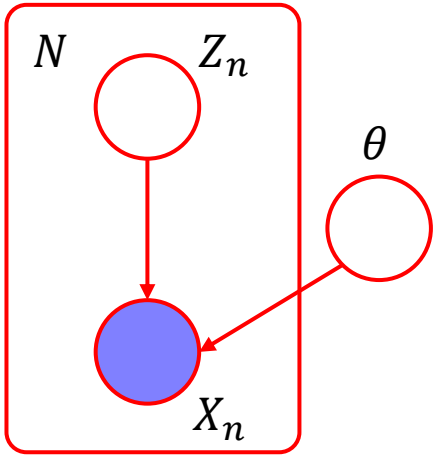
Variational AutoEncoder

- **Goal:** ensure there is one (or many) settings of Z_n to generate something very similar to each X_n in the dataset.

- To this end, let us define:

1. the prior $p(Z)$ over a high-dimensional space z, where samples of z can be easily drawn.

2. a family of deterministic functions $f(Z;\theta)$: $z \times \theta \rightarrow \mathcal{X}$.



Remarks: f is deterministic, but Z is random and θ is fixed, then $f(Z;\theta)$ is a random variable in the space $\mathcal{X}$.

Variational AutoEncoder

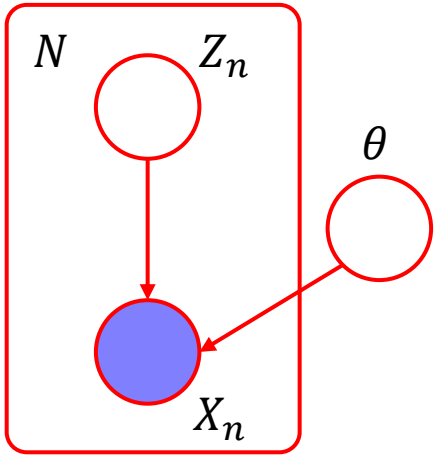
- **Objective:** optimize θ such that we can sample $z \sim p(Z)$ and, with high probability, $f(Z;\ \theta)$ will be like the X's in our dataset.
- This can be achieved by maximizing the likelihood, i.e,

$$p(X \mid \theta) = \int p(X \mid Z, \theta) p(Z) dZ,$$

where we define $p(X \mid Z, \theta)$ to be a Gaussian distribution, i.e.,

$$p(X \mid Z, \theta) = \mathcal{N}(X \mid f(Z; \theta), \sigma^2 I),$$

with mean $f(Z;\ \theta)$ and covariance equal to the identity matrix I times some scalar σ^2.

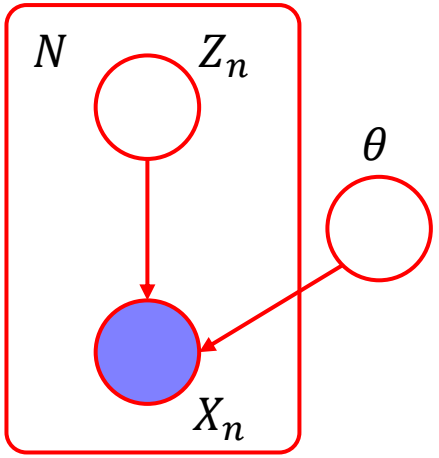


Variational AutoEncoder

- Two problems in solving the maximum likelihood:

$$p(X \mid \theta) = \int p(X \mid Z, \theta) p(Z) dZ$$

1. How to choose the latent variables Z such that we capture latent information?
2. How to deal with the integral over Z?

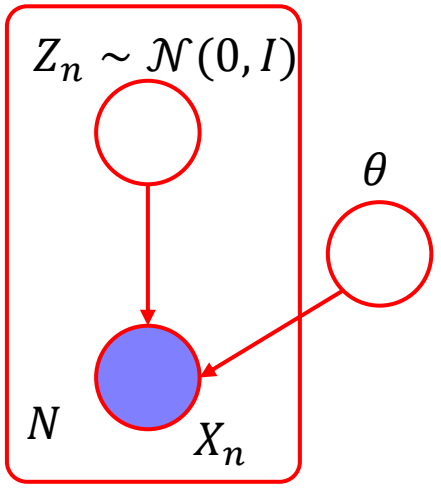


Choice of Latent Variable

- It is impossible to handcraft Z, since $z \sim p(Z)$ determines the highly complex image outputs X.
- VAEs assert that samples of Z can be drawn from a simple distribution, i.e., $p(Z) = \mathcal{N}(0, I)$, where I is the identity matrix.
- This is possible only if we learn $f(Z; \theta)$ in

$$p(X \mid Z, \theta) = \mathcal{N}(X \mid f(Z; \theta), \sigma^2 I)$$

with a powerful function approximator, e.g. a deep neural network!



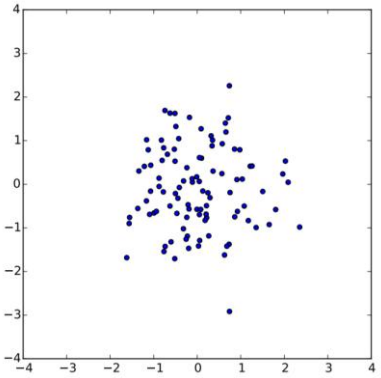
Choice of Latent Variable

Key idea: a sophisticated $f(Z;\theta)$ learned by deep learning can map any $z \sim \mathcal{N}(0, I)$ to X.

Example:

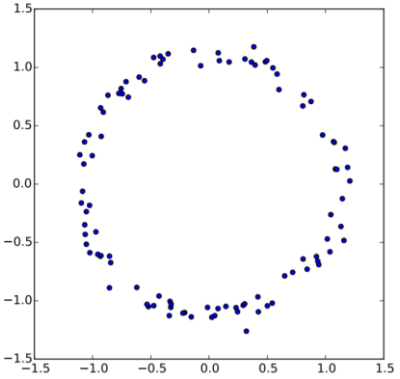
Given a random variable $z \sim \mathcal{N}(0, I)$, we can create another random variable $X = g(z)$ with a completely different distribution.

$z \sim \mathcal{N}(0, I)$



$$g(z) = \frac{z}{10} + \frac{z}{\|z\|}$$

$X = g(z)$



Choice of Latent Variable

Key idea: a sophisticated $f(Z;\theta)$ learned by deep learning can map any $z \sim \mathcal{N}(0, I)$ to X.

Remarks:

1. Let $f(Z;\ \theta)$ be a multi-layer neural network, where θ is the learnable parameters.
2. We can ensure $f(Z;\theta)$: $\mathcal{Z}\times\theta \rightarrow \mathcal{X}$ by considering $\ln p(\ X \mid Z, \theta\ )$ in the loss function, i.e.,

$$\begin{aligned}\ln p(\ X \mid Z, \theta\ ) &= \ln \mathcal{N}(X \mid f(Z;\theta), \sigma^2 I) \\ &= \ln\{\frac{1}{\sqrt{2\pi\sigma^2}}\exp -\frac{\big(X - f(Z;\theta)\big)^2}{\sigma^2}\} \\ &= -\|X - f(Z;\theta)\|_2^2 + \text{const.}\end{aligned}$$

L_2 loss term that forces outputs of $f(Z;\theta)$ to be close to X!

Maximum Log-Likelihood

- Now all that remains is to maximize the log-likelihood, i.e.,

$$\underset{\theta}{\operatorname{argmax}} \ln p(X \mid \theta) = \underset{\theta}{\operatorname{argmax}} \ln \int p(X \mid Z, \theta) p(Z) dZ,$$

where

$$p(Z) = \mathcal{N}(0, I), \quad \text{and}$$
$$p(X \mid Z, \theta) = \mathcal{N}(X \mid f(Z; \theta), \sigma^2 I).$$

- **A straightforward solution?** Approximate with log-likelihood with samples of $z \sim p(Z)$:

$$p(X) \approx \frac{1}{N} \sum_{i=1}^{N} p(X|f(z_i; \theta)) \ .$$

- **Problem:** X is in high dimensional spaces, N might need to be extremely large and $p(X|f(z_i; \theta)) \approx 0$ for most samples z.

Variational AutoEncoder

- **Solution:** sample Z that are likely to produce X, and compute $p(X)$ just from these samples.
- We define $q(Z \mid X)$ which takes a value of X and gives a distribution over Z values that are likely to produce X.
- To this end, we introduce $q(Z \mid X)$ into the log likelihood:

$$\begin{aligned}
\ln p(X) &= \sum_Z q(Z \mid X) \ln p(X) \\
&= \sum_Z q(Z \mid X) \ln \frac{p(X,Z)q(z|X)p(X)}{p(X,Z)q(z|X)} \\
&= \sum_Z q(Z \mid X) \ln \frac{p(X,Z)}{q(z|X)} + \sum_Z q(Z \mid X) \ln \frac{q(z|X)p(X)}{p(X,Z)} \\
&= \sum_Z q(Z \mid X) \ln \frac{p(X,Z)}{q(z|X)} + \sum_Z q(Z \mid X) \ln \frac{q(z|X)}{p(Z|X)}
\end{aligned}$$

Variational AutoEncoder

$$\ln p(X) = \underbrace{\sum_Z q(Z \mid X) \ln \frac{p(X,Z)}{q(z|X)}}_{\mathcal{L}(q,\theta)} + \underbrace{\sum_Z q(Z \mid X) \ln \frac{q(z|X)}{p(Z|X)}}_{KL[q(Z \mid X) \,\|\, p(Z \mid X)] \geq 0}$$

- Maximizing the log-likelihood is equivalent to maximizing the lower bound $\mathcal{L}(q,\theta)$ since the KL-divergence $KL[q(Z \mid X) \,\|\, p(Z \mid X)] \geq 0$.

Variational AutoEncoder

Expanding the lower bound term $\mathcal{L}(q, \theta)$, we get:

$$\sum_Z q(Z \mid X) \ln \frac{p(X,Z)}{q(z|X)}$$

$$= \sum_Z q(Z \mid X) \ln \frac{p(X|Z)p(Z)}{q(z|X)}$$

$$= \sum_Z q(Z \mid X) \ln p(X \mid Z) + \sum_Z q(Z \mid X) \ln \frac{p(Z)}{q(z|X)}$$

$$= \mathbb{E}_{z \sim q(Z|X)} \ln p(X \mid Z) - KL[q(Z \mid X) \parallel p(Z)]$$

VAE: Loss Function

- VAE's loss function is given by the negative lower bound, which we minimize using stochastic gradient descent.

$$\text{Loss} = -\mathbb{E}_{z\sim q(Z|X)} \ln p(X \mid Z) + KL[q(Z \mid X) \| p(Z)]$$

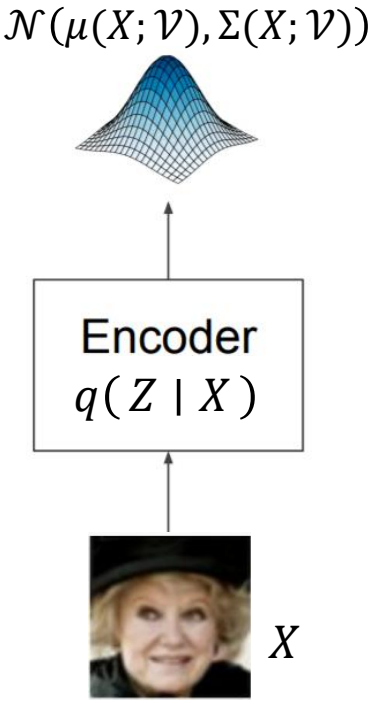
Decoder Encoder

- Now we can see the autoencoder, since $q(Z \mid X)$ is "encoding" X into Z, and $p(X \mid Z)$ is "decoding" it to reconstruct X.

VAE: Loss Function

$$\text{Loss} = -\mathbb{E}_{z\sim q(Z|X)} \ln p(X \mid Z) + KL[q(Z \mid X) \parallel p(Z)]$$

- Consider the second term of the loss function, we model the encoder $q(Z \mid X)$ with a neural network parameterized by $\mathcal{V}$.
- Assume Gaussian $q(Z \mid X) = \mathcal{N}(\mu(X;\mathcal{V}), \Sigma(X;\mathcal{V}))$, i.e., a neural network that outputs the mean $\mu(X)$, and diagonal covariance matrix $\Sigma(X;\mathcal{V})$.
- **Encoder:** Input is an Image X, output is a Gaussian distribution $\mathcal{N}(\mu(X;\mathcal{V}), \Sigma(X;\mathcal{V}))$.



VAE: Loss Function

$$\text{Loss} = -\mathbb{E}_{z\sim q(Z|X)} \ln p(X \mid Z) + KL[q(Z \mid X) \parallel p(Z)]$$

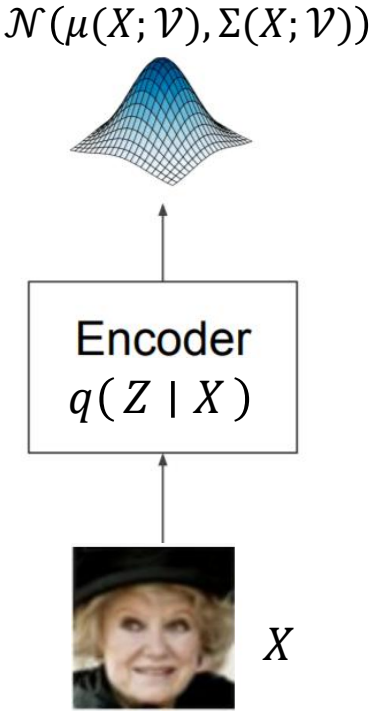
- Recall we defined $p(Z) = \mathcal{N}(0, I)$, the second loss term can now be rewritten as:

$$KL[\mathcal{N}(\mu(X;\mathcal{V}), \Sigma(X;\mathcal{V})) \parallel \mathcal{N}(0, I)]$$

$$= \frac{1}{2}(\text{trace}(\Sigma(X;\mathcal{V})) + (\mu(X;\mathcal{V}))^{\top}(\mu(X;\mathcal{V})) -$$

$$k - \log\det(\Sigma(X;\mathcal{V})),$$

where k is the dimensionality of the distribution.



- **Insight**: This loss forces the latent space to be close to the normal distribution!

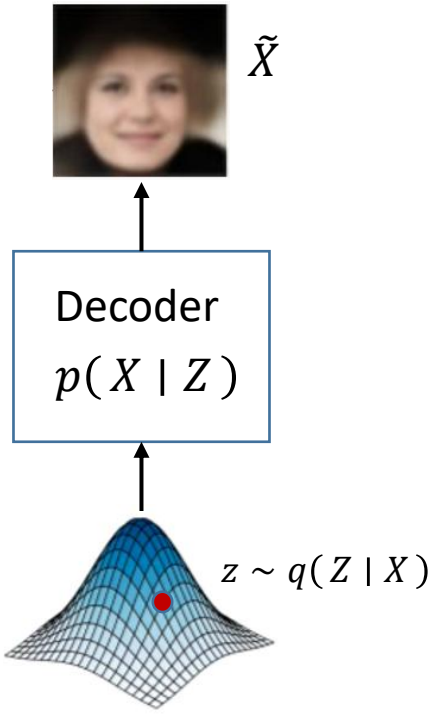
VAE: Loss Function

$$\text{Loss} = \boxed{-\mathbb{E}_{z\sim q(Z|X)} \ln p(X \mid Z)} + KL[q(Z \mid X) \parallel p(Z)]$$

- Consider the first term of the loss function, recall that we model the decoder $p(X \mid Z)$ as:

$$p(X \mid Z, \theta) = \mathcal{N}(X \mid f(Z;\theta), \sigma^2 I)$$

- where $f(Z;\theta)$ is a learnable neural network parameterized by θ.
- **Decoder:** Input is a sample from the latent space Z, output is the reconstructed image $\tilde{X} = f(Z;\theta)$.



VAE: Loss Function

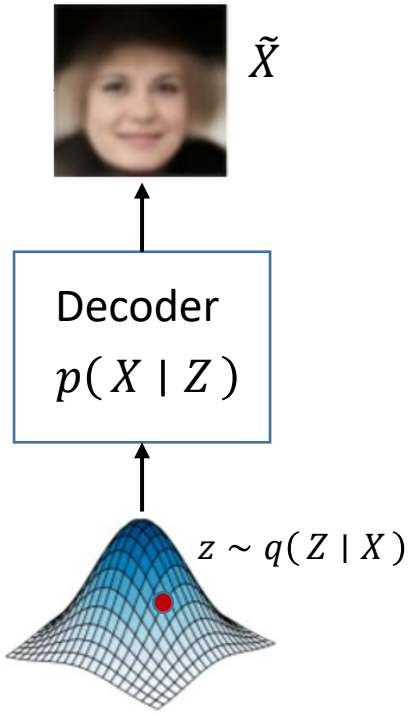
$$\text{Loss} = \boxed{-\mathbb{E}_{z\sim q(Z|X)} \ln p(X \mid Z)} + KL[q(Z \mid X) \,\|\, p(Z)]$$

- This leads to the square loss as mentioned earlier:

$$-\mathbb{E}_{z\sim q(Z|X)} \ln p(X \mid Z) = -\mathbb{E}_{z\sim q(Z|X)} \ln \mathcal{N}(X \mid f(Z;\theta), \sigma^2 I)$$

$$= -\mathbb{E}_{z\sim q(Z|X)} [\ln \left\{ \frac{1}{\sqrt{2\pi\sigma^2}} \exp - \frac{\left(X - f(Z;\theta)\right)^2}{\sigma^2} \right\}]$$

$$= \mathbb{E}_{z\sim q(Z|X)} \|X - \underbrace{f(Z;\theta)}_{\tilde{X}}\|_2^2 + \text{const.}$$



- This is the expected image reconstruction loss!

VAE: Optimizing the Loss Function

- Putting the loss terms together, we get:

$$\begin{aligned}\text{Loss} &= -\mathbb{E}_{z\sim q(Z|X)} \ln p(X \mid Z) + KL[q(Z \mid X) \parallel p(Z)] \\ &= \mathbb{E}_{z\sim q(Z|X)} \|X - f(Z;\theta)\|_2^2 + KL[\mathcal{N}(\mu(X), \Sigma(X) \parallel \mathcal{N}(0, I)]\end{aligned}$$

- **An easy approach?** We can optimize the loss by computing the gradient over every sample $X \sim \mathcal{D}$ and $z \sim Q(Z \mid X)$.

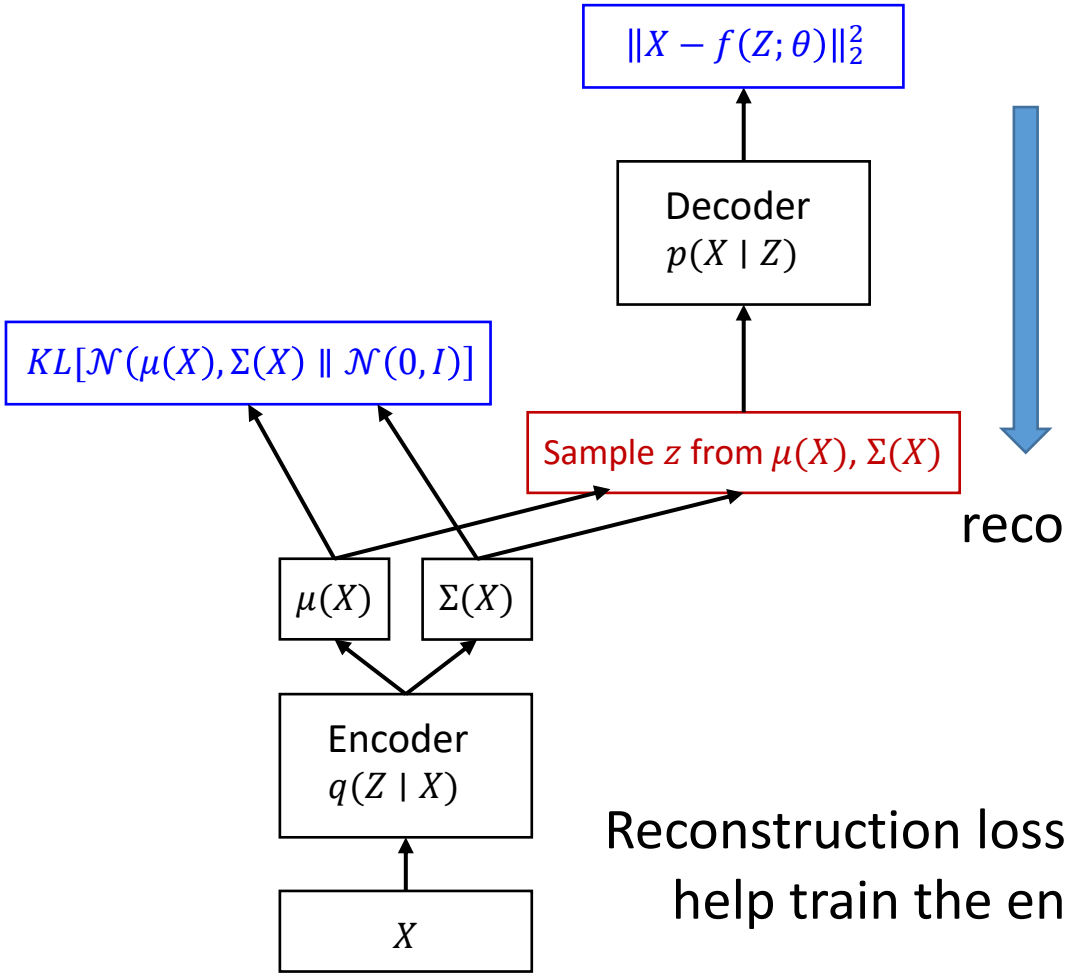
$$\text{Loss} = \mathbb{E}_{X\sim\mathcal{D}} \left\{ -\mathbb{E}_{z\sim q(Z|X)} \ln p(X \mid Z) + KL[q(Z \mid X) \parallel p(Z)] \right\}$$

- **Problem:** Gradient is now independent of encoder!

$$\text{Gradient} = \nabla(\underbrace{\log p(X \mid Z)}_{} - KL[q(Z \mid X) \parallel p(Z)])$$

$q(Z \mid X)$ is not trained!

VAE: Optimizing the Loss Function



Gradient from the reconstruction loss stops here!

Reconstruction loss does not help train the encoder!

VAE: Optimizing the Loss Function

Reparameterization Trick

- **Solution:** Move the sampling to an input layer!
- Given the mean $\mu(X)$ and covariance $\Sigma(X)$ of $q(Z \mid X)$, we can sample from $\mathcal{N}(\mu(X), \Sigma(X))$ by:
  1. sampling $\varepsilon \sim \mathcal{N}(0, I)$, then
  2. compute $z = \mu(X) + \Sigma^{1/2}(X) * \varepsilon$
- The loss and gradient become:

Kept fixed

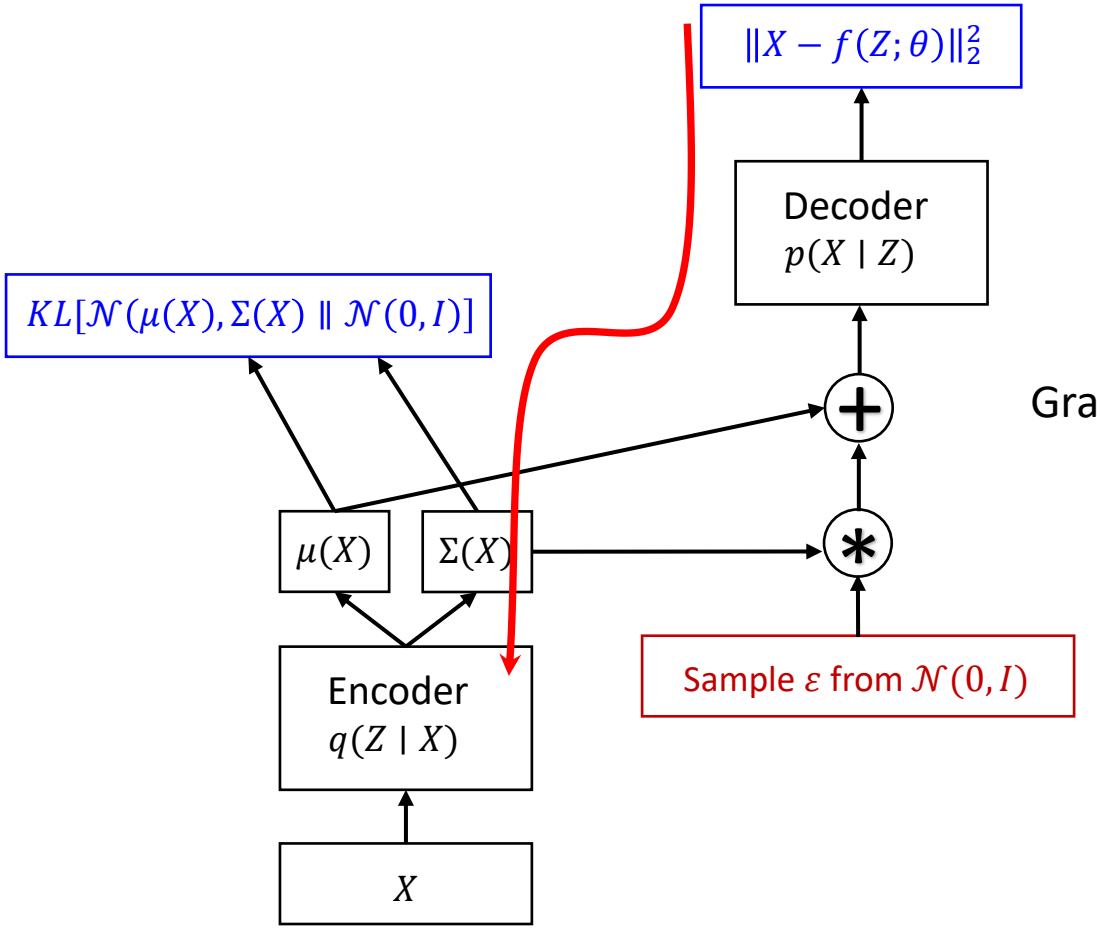
$$\text{Loss} = -\mathbb{E}_{X\sim\mathcal{D}}\{-\mathbb{E}_{\varepsilon\sim\mathcal{N}(0,I)} \ln p(X \mid z = \mu(X) + \Sigma^{1/2}(X) * \varepsilon) + KL[q(Z \mid X) \parallel p(Z)]\}$$

$$\text{Gradient} = \nabla(\log p(X \mid \underbrace{z = \mu(X) + \Sigma^{1/2}(X) * \varepsilon}) - KL[q(Z \mid X) \parallel p(Z)])$$

function of $q(Z \mid X)$!

VAE: Optimizing the Loss Function

Reparameterization Trick



Gradient of the reconstruction loss flows to $q(Z \mid X)$!

VAE Training

Given: a dataset of examples $X = \{X_1, X_2, \dots\}$.

1. Initialize parameters for Encoder and Decoder
2. Repeat till convergence:

   i. $X^M \leftarrow$ Random minibatch of M examples from X

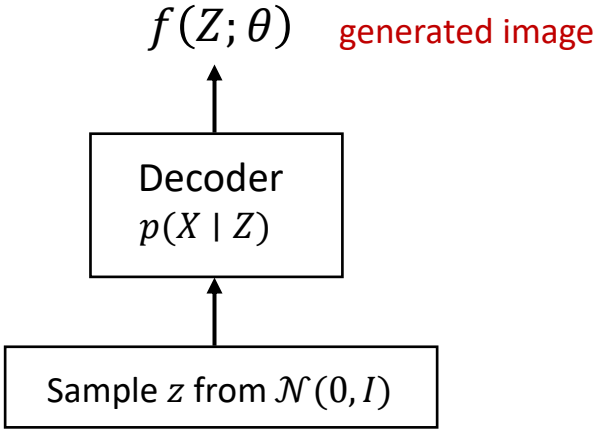
   ii. $\varepsilon \leftarrow$ Sample M noise vectors from $\mathcal{N}(0, I)$

   iii. Compute $L(X^M, \varepsilon, \theta)$ (i.e. run a forward pass in the neural network)

   iv. Gradient descent on L to update the Encoder and Decoder (backpropagation).

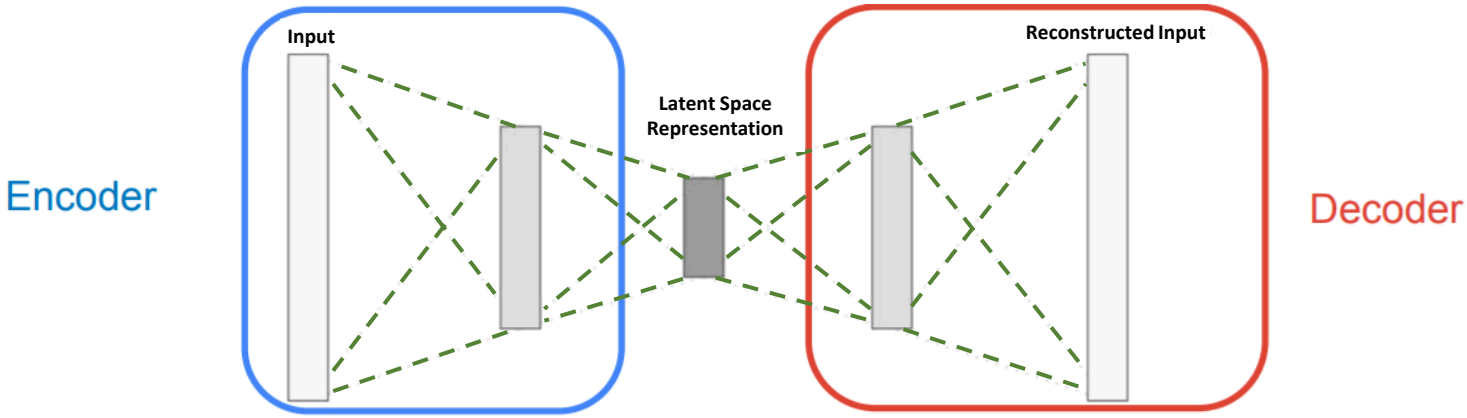
VAE Testing

- At test-time, we want to evaluate the performance of VAE to generate a new sample.
- Remove the Encoder since no test-image for the generation task.
- Sample $z \sim \mathcal{N}(0, I)$ and pass it through the Decoder.
- No good quantitative metric, relies on visual inspection.

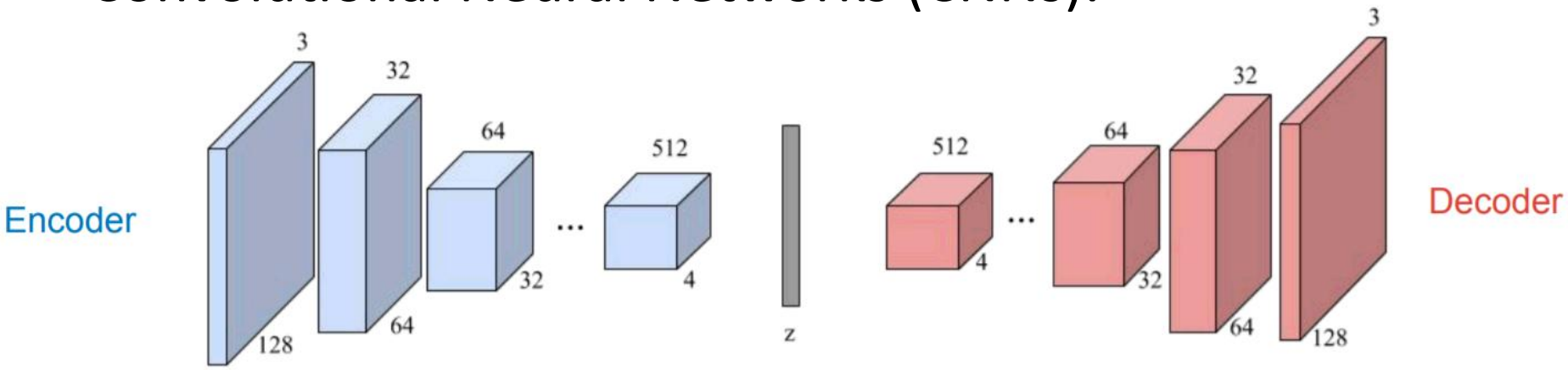


Common VAE Architecture

- Multi-Layer Perceptrons (MLPs):



- Convolutional Neural Networks (CNNs):



Disentangle latent factor

- Visualizations of learned data manifold for generative models with two-dimensional latent space.
- We can see that the generated images are distributed according to the disentangled latent factor.



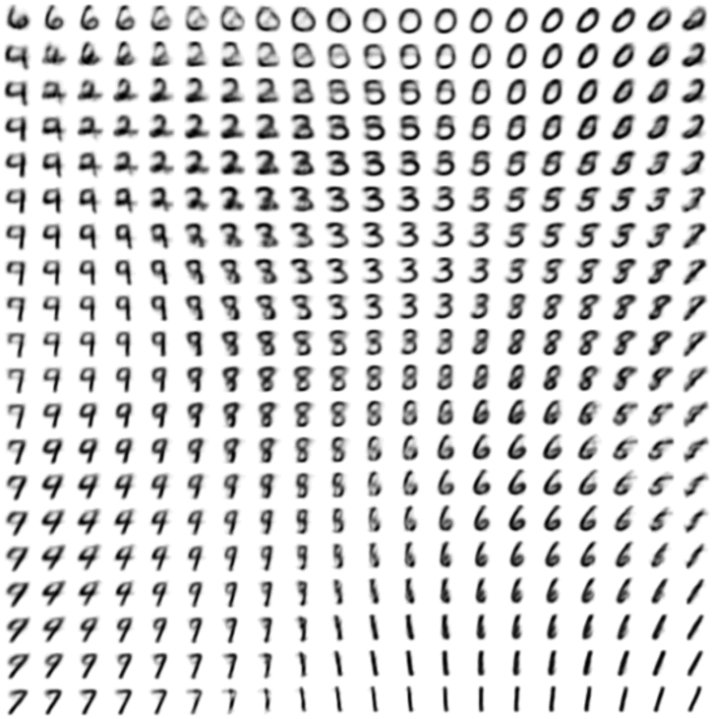


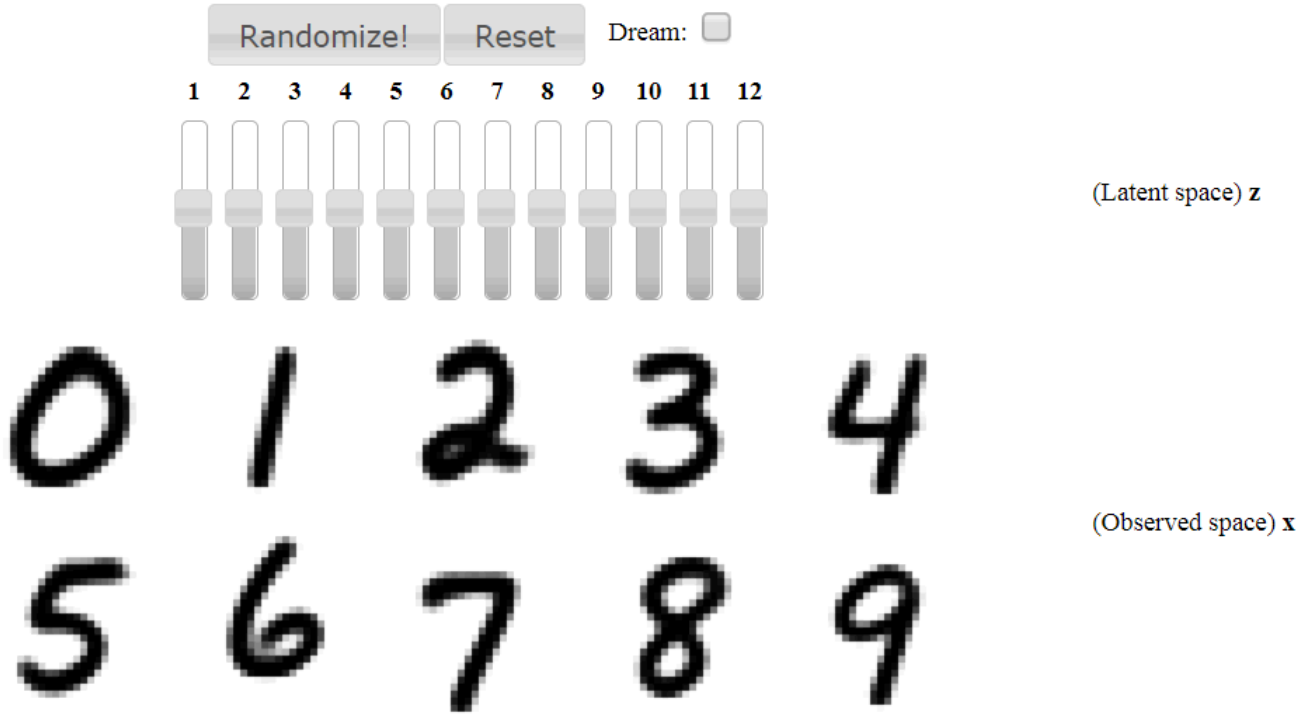
Image Source: Kingmal and Welling, Auto-Encoding Variational Bayes, ICLR'14

Disentangle latent factor

Digit Fantasies by a Deep Generative Model

Instructions:

1. Dream mode: check 'dream' to let the model fantasize digits.
2. Alternatively, you can wiggle the sliders yourselves to wander through **z**-space and observe the effects in **x**-space.



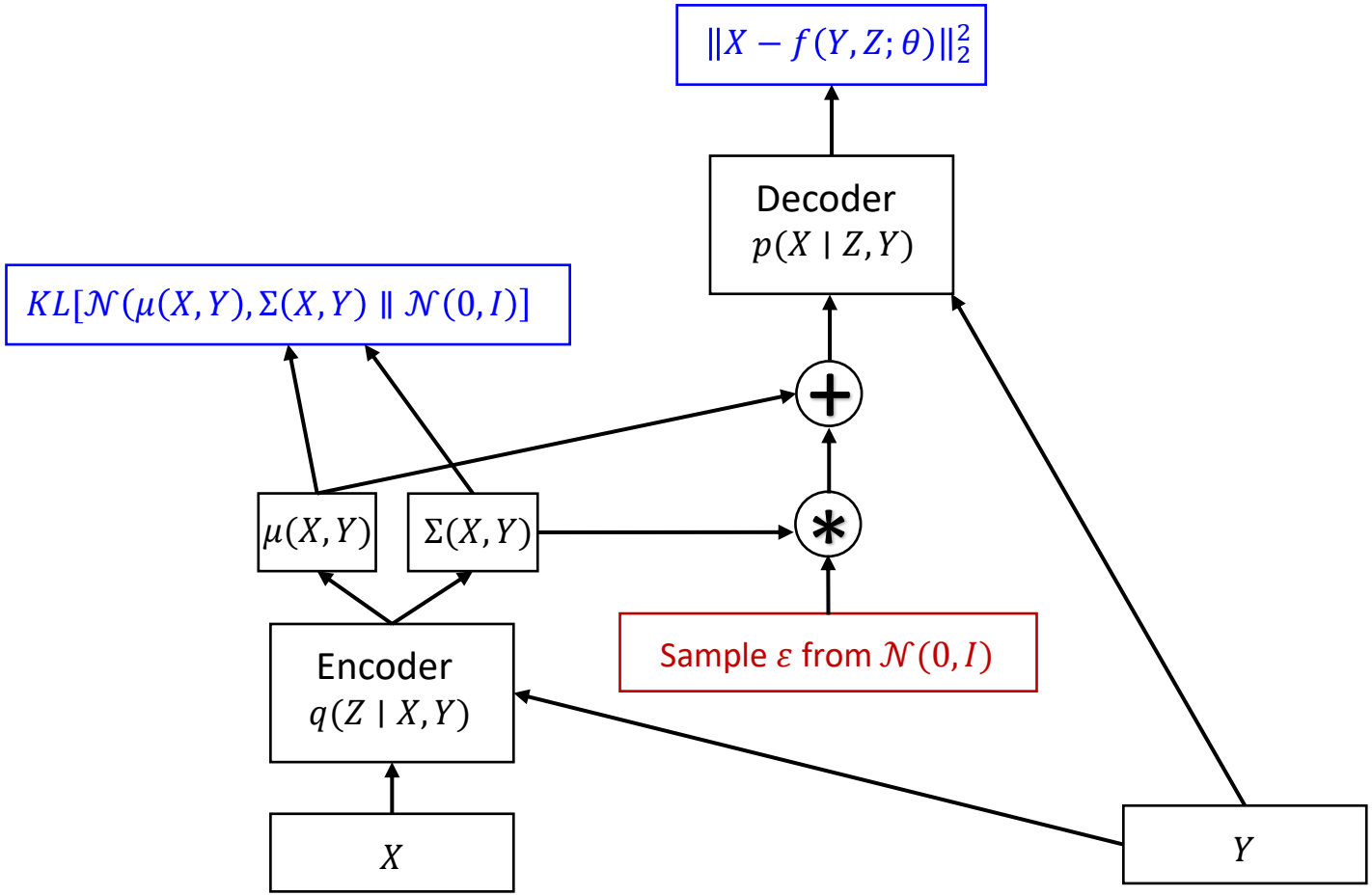
http://www.dpkingma.com/sgvb_mnist_demo/demo.html

Conditional VAE (CVAE)

- What if we have labels? (e.g. digit labels or attributes) Or other inputs we wish to condition on (Y).
- None of the derivation changes:

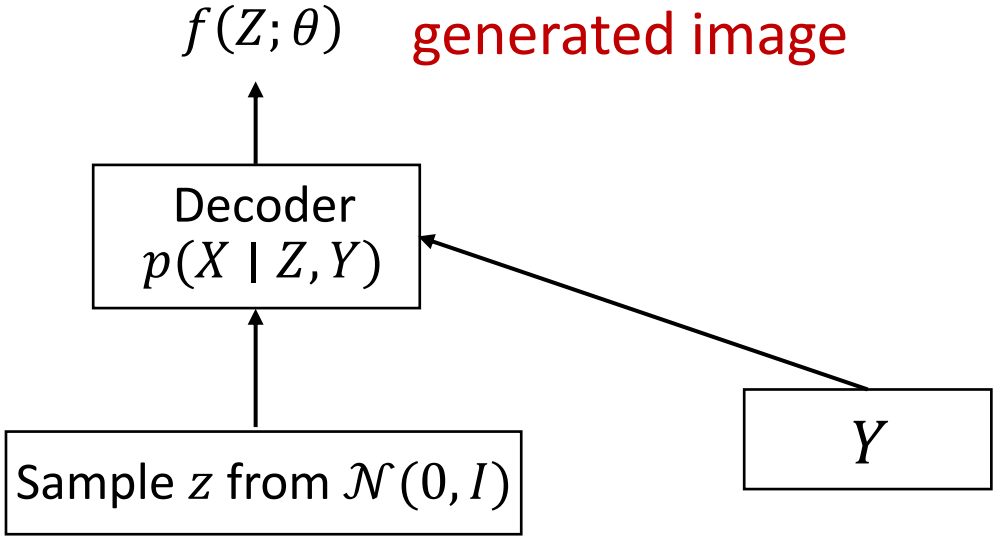
1. Replace all $p(X \mid Z)$ with $p(X \mid Z, Y)$
2. Replace all $q(Z \mid X)$ with $q(Z \mid X, Y)$
3. Go through the same KL-divergence procedure, to get the same lower bound.

Conditional VAE (CVAE)



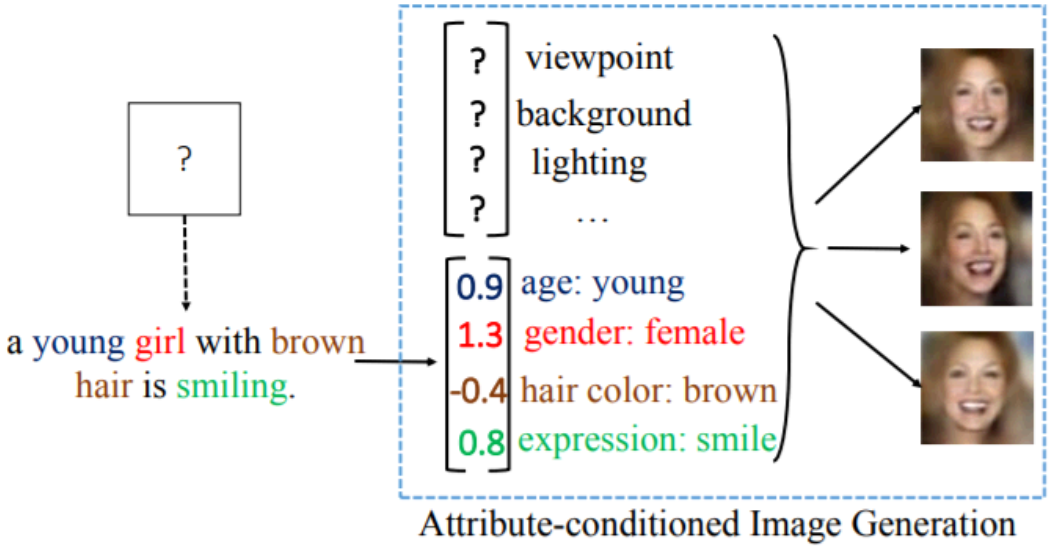
Conditional VAE (CVAE)

- Remove the Encoder at test time.
- Sample $z \sim \mathcal{N}(0, I)$ and input a desired Y to the Decoder.



CVAE Example: Conditioned Image Generation from Visual Attributes

- A vector of visual attributes is extracted from a natural language description.
- This attribute vector is then combined with learned latent factors to generate diverse image samples.

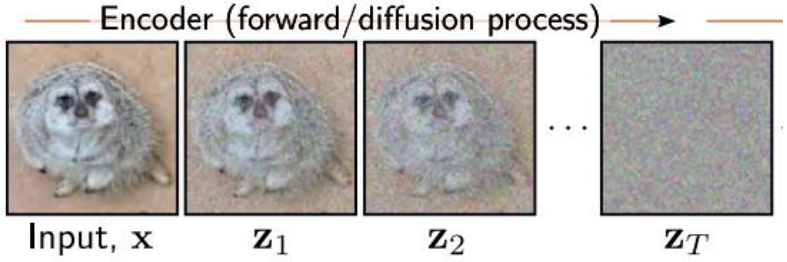


Xinchen Yan, Jimei Yang, Kihyuk Sohn, Honglak Lee, Attribute2Image: Conditional Image Generation from Visual Attributes, ECCV 2016

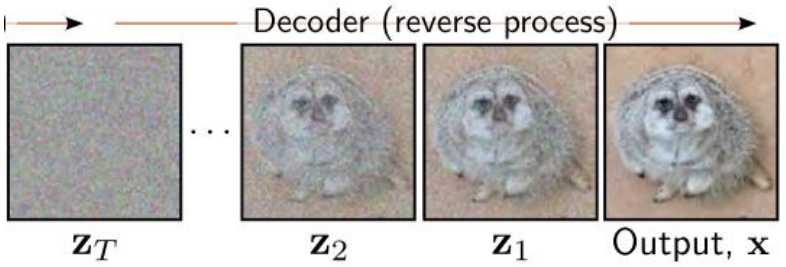
Diffusion Models

- A diffusion model consists of an encoder and a decoder for stochastic mappings.

- **Encoder:** Takes a data sample $\mathbf{x}$ and maps it through a series of intermediate latent variables $\mathbf{z}_1 \ldots \mathbf{z}_T$.



- **Decoder:** Starts with $\mathbf{z}_T$ and maps back through $\mathbf{z}_{T-1}, \ldots, \mathbf{z}_1$ until finally (re-)creates a data point $\mathbf{x}$.



Diffusion Models

- **Encoder (No learning):** Gradually blends the input with samples of white noise.

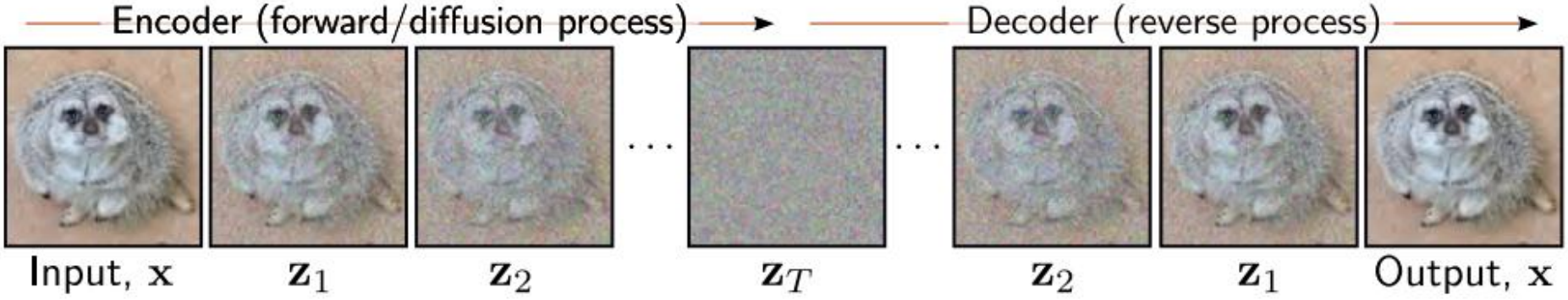
  Conditional distribution $q(\mathbf{z}_T \mid \mathbf{x})$ and marginal distribution $q(\mathbf{z}_T)$ of final latent variable eventually become the standard normal distribution.

- **Decoder (Deep Networks):** All the learned parameters are in the decoder.

  A series of networks are trained to map backward between each adjacent pair of latent variables $\mathbf{z}_t$ and $\mathbf{z}_{t-1}$.

Diffusion Models

- **Loss function:** Encourages each network to invert the corresponding encoder step.
- Noise is gradually removed from the representation until a realistic-looking data example remains.
- **Generate a new data (Inference):** Draw a sample from $q(\mathbf{z}_T)$ and pass it through the decoder to get $\mathbf{x}$.



Encoder (Forward Process)

- **Diffusion** or **forward process:** Maps a data example $\mathbf{x}$ through a series of intermediate variables $\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_T$ with the same size as $\mathbf{x}$ according to:

$$\begin{aligned} \mathbf{z}_1 &= \sqrt{1-\beta_1} \cdot \mathbf{x} + \sqrt{\beta_1} \cdot \boldsymbol{\epsilon}_1 \\ \mathbf{z}_t &= \sqrt{1-\beta_t} \cdot \mathbf{z}_{t-1} + \sqrt{\beta_t} \cdot \boldsymbol{\epsilon}_t \qquad \forall\, t \in 2, \ldots, T, \end{aligned}$$

- $\boldsymbol{\epsilon}_t$ is noise drawn from a standard normal distribution.
- Hyperparameters $\beta_t \in [0, 1]$ determine how quickly the noise is blended and known as the noise schedule.

Encoder (Forward Process)

- **Forward process:** can equivalently be written as:

$$
\begin{aligned}
q(\mathbf{z}_1|\mathbf{x}) &= \text{Norm}_{\mathbf{z}_1}\left[\sqrt{1-\beta_1}\mathbf{x}, \beta_1\mathbf{I}\right] \\
q(\mathbf{z}_t|\mathbf{z}_{t-1}) &= \text{Norm}_{\mathbf{z}_t}\left[\sqrt{1-\beta_t}\mathbf{z}_{t-1}, \beta_t\mathbf{I}\right] \qquad \forall\, t \in 2, \ldots, T.
\end{aligned}
$$

- With sufficient steps T, all traces of original data are removed, and $q(\mathbf{z}_T \mid \mathbf{x}) = q(\mathbf{z}_T)$ becomes a standard normal distribution.

- The joint distribution of all the latent variables $\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_T$ given input $\mathbf{x}$ is:

$$
q(\mathbf{z}_{1\ldots T}|\mathbf{x}) = q(\mathbf{z}_1|\mathbf{x}) \prod_{t=2}^{T} q(\mathbf{z}_t|\mathbf{z}_{t-1}).
$$

Encoder: Diffusion Kernel $q(\mathbf{z}_t \mid \mathbf{x})$

- To train the decoder to invert this process, we use multiple samples $\mathbf{z}_t$ at time t for the same example $\mathbf{x}$, i.e.:

$$z_t \sim q(z_t \mid z_{t-1})q(z_{t-2} \mid z_{t-3}) \ldots q(z_1 \mid \mathbf{x})$$

- **Problem:** However, generating these sequentially using is time-consuming!
- **Solution:** Use a closed-form expression for $q(\mathbf{z}_t \mid \mathbf{x})$ to directly draw samples $\mathbf{z}_t$ from $\mathbf{x}$ without computing the intermediate variables.

Encoder: Diffusion Kernel $q(\mathbf{z}_t \mid \mathbf{x})$

Derive closed-form expression for $q(\mathbf{z}_t \mid \mathbf{x})$:

- Consider the first two steps of the forward process:

$$\begin{aligned} \mathbf{z}_1 &= \sqrt{1-\beta_1} \cdot \mathbf{x} + \sqrt{\beta_1} \cdot \boldsymbol{\epsilon}_1 \\ \mathbf{z}_2 &= \sqrt{1-\beta_2} \cdot \mathbf{z}_1 + \sqrt{\beta_2} \cdot \boldsymbol{\epsilon}_2. \end{aligned}$$

- Substituting the first equation into the second, we get:

$$\mathbf{z}_2 = \sqrt{(1-\beta_2)(1-\beta_1)} \cdot \mathbf{x} + \sqrt{1-(1-\beta_2)(1-\beta_1)} \cdot \boldsymbol{\epsilon},$$

- ϵ is also a sample from a standard normal distribution.

Encoder: Diffusion Kernel $q(\mathbf{z}_t \mid \mathbf{x})$

Derive closed-form expression for $q(\mathbf{z}_t \mid \mathbf{x})$ (cont.):

- Continuing the process, we will get:

$$\mathbf{z}_t = \sqrt{\alpha_t} \cdot \mathbf{x} + \sqrt{1-\alpha_t} \cdot \boldsymbol{\epsilon}, \qquad \alpha_t = \prod_{s=1}^{t} 1 - \beta_s$$

- We can equivalently write this in probabilistic form:

$$q(\mathbf{z}_t|\mathbf{x}) = \mathrm{Norm}_{\mathbf{z}_t}\left[\sqrt{\alpha_t} \cdot \mathbf{x}, (1-\alpha_t)\mathbf{I}\right].$$

- Easy to generate samples from $q(\mathbf{z}_t \mid \mathbf{x})$ since we do not care about intermediate latent variables.

Decoder Model (Reverse Process)

- Learning a diffusion model $\Longrightarrow$ learning the reverse process.

- i.e., learn a series of probabilistic mappings back from latent variable $\mathbf{z}_T \rightarrow \mathbf{z}_{T-1} \rightarrow \mathbf{z}_{T-2} \ldots \rightarrow \mathbf{x}$.

- True reverse distributions $q(\mathbf{z}_{t-1} \mid \mathbf{z}_t)$ of diffusion process are complex multi-modal distributions that depend on the data distribution $\Pr(\mathbf{x})$.

Decoder Model (Reverse Process)

- We approximate these as normal distributions:

$$\begin{aligned} Pr(\mathbf{z}_T) &= \text{Norm}_{\mathbf{z}_T}[\mathbf{0}, \mathbf{I}] \\ Pr(\mathbf{z}_{t-1}|\mathbf{z}_t, \boldsymbol{\phi}_t) &= \text{Norm}_{\mathbf{z}_{t-1}}\Big[\mathbf{f}_t[\mathbf{z}_t, \boldsymbol{\phi}_t], \sigma_t^2\mathbf{I}\Big] \\ Pr(\mathbf{x}|\mathbf{z}_1, \boldsymbol{\phi}_1) &= \text{Norm}_{\mathbf{x}}\Big[\mathbf{f}_1[\mathbf{z}_1, \boldsymbol{\phi}_1], \sigma_1^2\mathbf{I}\Big], \end{aligned}$$

- $f_t[\mathbf{z}_t, \phi_1]$: neural network that computes the mean of the normal distribution in the estimated mapping $\mathbf{z}_t \rightarrow \mathbf{z}_{t-1}$.

- The terms $\{\sigma_t^2\}$ predetermined.

- Normal approximation is reasonable if hyper-parameters β_t in the diffusion process are close to zero (and the number of time steps T is large).

Decoder Model (Reverse Process)

- **Inference:** We generate new examples from $Pr(\mathbf{x})$ using ancestral sampling:
  1. Start by drawing $\mathbf{z}_T \sim Pr(\mathbf{z}_T)$, and then
  2. Sample $\mathbf{z}_{T-1} \sim Pr(\mathbf{z}_{T-1} \mid \mathbf{z}_T, \phi_T)$, and so on
  3. Until we finally generate $\mathbf{x}$ from $Pr(\mathbf{x} \mid \mathbf{z}_1, \phi_1)$.

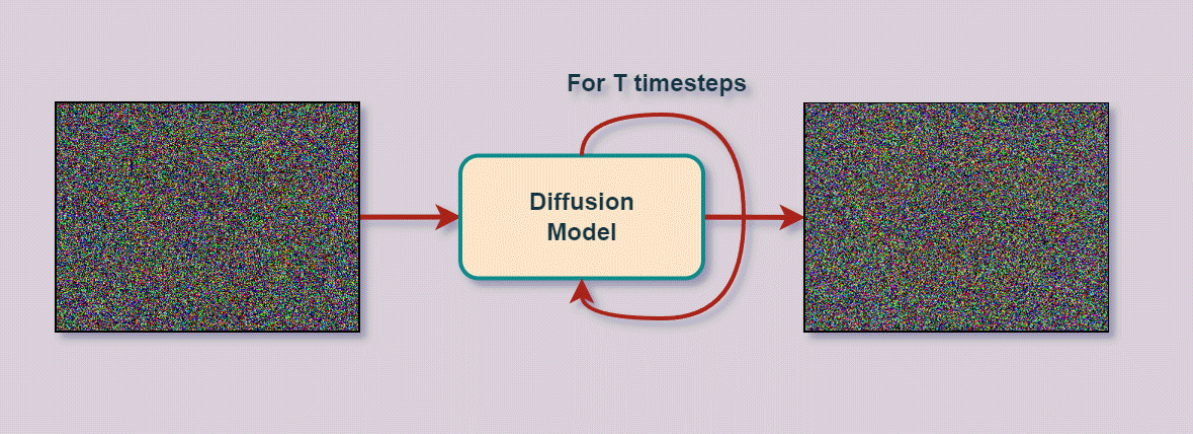


Image Source: https://learnopencv.com/image-generation-using-diffusion-models/

Training: Maximum Log-likelihood

- The joint distribution of the observed variable $\mathbf{x}$ and the latent variables $\{\mathbf{z}_t\}$ is:

$$Pr(\mathbf{x}, \mathbf{z}_{1...T}|\boldsymbol{\phi}_{1...T}) = Pr(\mathbf{x}|\mathbf{z}_1, \boldsymbol{\phi}_1) \prod_{t=2}^{T} Pr(\mathbf{z}_{t-1}|\mathbf{z}_t, \boldsymbol{\phi}_t) \cdot Pr(\mathbf{z}_T).$$

- The likelihood of the observed data $\Pr(\mathbf{x} \mid \boldsymbol{\phi}_{1...T})$ is found by marginalizing over the latent variables:

$$Pr(\mathbf{x}|\boldsymbol{\phi}_{1...T}) = \int Pr(\mathbf{x}, \mathbf{z}_{1...T}|\boldsymbol{\phi}_{1...T}) d\mathbf{z}_{1...T}.$$

- To train the model, we maximize the log-likelihood of the training data $\{\mathbf{x}_i\}$ with respect to the parameters $\boldsymbol{\phi}$:

$$\hat{\boldsymbol{\phi}}_{1...T} = \underset{\boldsymbol{\phi}_{1...T}}{\operatorname{argmax}} \left[\sum_{i=1}^{I} \left[\log \left[Pr(\mathbf{x}_i|\boldsymbol{\phi}_{1...T}) \right] \right] \right].$$

Evidence Lower Bound (ELBO)

- We cannot maximize the log-likelihood because the marginalization in $\Pr(\mathbf{x} \mid \phi_{1\ldots T})$ is intractable.
- Use lower bound on the likelihood and optimize over $\phi_{1\ldots T}$, we derive the lower bound using the encoder distribution $q(\mathbf{z}_{1\ldots T} \mid \mathbf{x})$

$$
\begin{aligned}
\log\left[Pr(\mathbf{x}|\boldsymbol{\phi}_{1\ldots T})\right] &= \log\left[\int Pr(\mathbf{x}, \mathbf{z}_{1\ldots T}|\boldsymbol{\phi}_{1\ldots T}) d\mathbf{z}_{1\ldots T}\right] \\
&= \log\left[\int q(\mathbf{z}_{1\ldots T}|\mathbf{x}) \frac{Pr(\mathbf{x}, \mathbf{z}_{1\ldots T}|\boldsymbol{\phi}_{1\ldots T})}{q(\mathbf{z}_{1\ldots T}|\mathbf{x})} d\mathbf{z}_{1\ldots T}\right] \\
&\geq \int q(\mathbf{z}_{1\ldots T}|\mathbf{x}) \log\left[\frac{Pr(\mathbf{x}, \mathbf{z}_{1\ldots T}|\boldsymbol{\phi}_{1\ldots T})}{q(\mathbf{z}_{1\ldots T}|\mathbf{x})}\right] d\mathbf{z}_{1\ldots T}.
\end{aligned}
$$

lower bound

Simplifying the ELBO

- The ELBO can be written into:

$$\begin{aligned}
&\text{ELBO}\left[\boldsymbol{\phi}_{1\ldots T}\right] \\
&\quad = \int q(\mathbf{z}_{1\ldots T}|\mathbf{x}) \log\left[\frac{Pr(\mathbf{x}, \mathbf{z}_{1\ldots T}|\boldsymbol{\phi}_{1\ldots T})}{q(\mathbf{z}_{1\ldots T}|\mathbf{x})}\right] d\mathbf{z}_{1\ldots T} \\
&\quad \approx \int q(\mathbf{z}_{1\ldots T}|\mathbf{x}) \left(\log\left[Pr(\mathbf{x}|\mathbf{z}_1, \boldsymbol{\phi}_1)\right] + \sum_{t=2}^{T} \log\left[\frac{Pr(\mathbf{z}_{t-1}|\mathbf{z}_t, \boldsymbol{\phi}_t)}{q(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{x})}\right]\right) d\mathbf{z}_{1\ldots T}
\end{aligned}$$

Simplifying the ELBO

Proof:

$$\text{ELBO}[\boldsymbol{\phi}_{1\ldots T}] = \int q(\mathbf{z}_{1\ldots T}|\mathbf{x}) \log \left[\frac{Pr(\mathbf{x}, \mathbf{z}_{1\ldots T}|\boldsymbol{\phi}_{1\ldots T})}{q(\mathbf{z}_{1\ldots T}|\mathbf{x})} \right] d\mathbf{z}_{1\ldots T}.$$

Markov Chains

$$\log \left[\frac{Pr(\mathbf{x}, \mathbf{z}_{1\ldots T}|\boldsymbol{\phi}_{1\ldots T})}{q(\mathbf{z}_{1\ldots T}|\mathbf{x})} \right] = \log \left[\frac{Pr(\mathbf{x}|\mathbf{z}_1, \boldsymbol{\phi}_1) \prod_{t=2}^{T} Pr(\mathbf{z}_{t-1}|\mathbf{z}_t, \boldsymbol{\phi}_t) \cdot Pr(\mathbf{z}_T)}{q(\mathbf{z}_1|\mathbf{x}) \prod_{t=2}^{T} q(\mathbf{z}_t|\mathbf{z}_{t-1})} \right]$$

$$= \log \left[\frac{Pr(\mathbf{x}|\mathbf{z}_1, \boldsymbol{\phi}_1)}{q(\mathbf{z}_1|\mathbf{x})} \right] + \log \left[\frac{\prod_{t=2}^{T} Pr(\mathbf{z}_{t-1}|\mathbf{z}_t, \boldsymbol{\phi}_t)}{\prod_{t=2}^{T} q(\mathbf{z}_t|\mathbf{z}_{t-1})} \right] + \log \left[Pr(\mathbf{z}_T) \right].$$

Bayes' Rule

Conditional Diffusion Distribution:

$$q(\mathbf{z}_t|\mathbf{z}_{t-1}) = q(\mathbf{z}_t|\mathbf{z}_{t-1}, \mathbf{x}) = \frac{q(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{x}) q(\mathbf{z}_t|\mathbf{x})}{q(\mathbf{z}_{t-1}|\mathbf{x})},$$

Conditional Independence

Simplifying the ELBO

Proof (cont.):

Forward process $q(z_T \mid x)$ and $p(z_T)$ are approximated by standard normal distribution

$$\log\left[\frac{Pr(\mathbf{x}, \mathbf{z}_{1\ldots T}|\boldsymbol{\phi}_{1\ldots T})}{q(\mathbf{z}_{1\ldots T}|\mathbf{x})}\right]$$

$$= \log\left[\frac{Pr(\mathbf{x}|\mathbf{z}_1, \boldsymbol{\phi}_1)}{q(\mathbf{z}_1|\mathbf{x})}\right] + \log\left[\frac{\prod_{t=2}^{T} Pr(\mathbf{z}_{t-1}|\mathbf{z}_t, \boldsymbol{\phi}_t) \cdot q(\mathbf{z}_{t-1}|\mathbf{x})}{\prod_{t=2}^{T} q(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{x}) \cdot q(\mathbf{z}_t|\mathbf{x})}\right] + \log\left[Pr(\mathbf{z}_T)\right]$$

$$= \log\left[Pr(\mathbf{x}|\mathbf{z}_1, \boldsymbol{\phi}_1)\right] + \log\left[\frac{\prod_{t=2}^{T} Pr(\mathbf{z}_{t-1}|\mathbf{z}_t, \boldsymbol{\phi}_t)}{\prod_{t=2}^{T} q(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{x})}\right] + \log\left[\frac{Pr(\mathbf{z}_T)}{q(\mathbf{z}_T|\mathbf{x})}\right]$$

$$\approx \log\left[Pr(\mathbf{x}|\mathbf{z}_1, \boldsymbol{\phi}_1)\right] + \sum_{t=2}^{T} \log\left[\frac{Pr(\mathbf{z}_{t-1}|\mathbf{z}_t, \boldsymbol{\phi}_t)}{q(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{x})}\right],$$

≈ 0

$$\log\left[\frac{p(x \mid z_1, \phi_1)}{q(z_1 \mid x)} \cdot \frac{p(z_1 \mid z_2, \phi_2) q(z_1 \mid x) p(z_2 \mid z_3, \phi_3) q(z_2 \mid x) \ldots p(z_{T-1} \mid z_T, \phi_T) q(z_{T-1} \mid x)}{q(z_1 \mid z_2, x) q(z_2 \mid x) q(z_2 \mid z_3, x) q(z_3 \mid x) \ldots q(z_{T-1} \mid z_T, x) q(z_T \mid x)} \cdot p(z_T)\right]$$

$$= \log p(x \mid z_1, \phi_1) + \log \frac{\prod_{t=2}^{T} p(z_{t-1}|z_t, \phi_t)}{\prod_{t=2}^{T} p(z_{t-1}|z_t, x)} + \log \frac{p(z_T)}{q(z_T|x)}$$

■

Conditional Diffusion Distribution $q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x})$

- To compute an expression for $q(z_{t-1} \mid z_t, x)$ we start with Bayes' rule:

$$
\begin{aligned}
q(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{x}) &= \frac{q(\mathbf{z}_t|\mathbf{z}_{t-1}, \mathbf{x})q(\mathbf{z}_{t-1}|\mathbf{x})}{q(\mathbf{z}_t|\mathbf{x})} \\
&\propto q(\mathbf{z}_t|\mathbf{z}_{t-1})q(\mathbf{z}_{t-1}|\mathbf{x}) \\
&= \text{Norm}_{\mathbf{z}_t}\left[\sqrt{1-\beta_t}\cdot\mathbf{z}_{t-1}, \beta_t\mathbf{I}\right]\text{Norm}_{\mathbf{z}_{t-1}}\left[\sqrt{\alpha_{t-1}}\cdot\mathbf{x}, (1-\alpha_{t-1})\mathbf{I}\right] \\
&\propto \text{Norm}_{\mathbf{z}_{t-1}}\left[\frac{1}{\sqrt{1-\beta_t}}\mathbf{z}_t, \frac{\beta_t}{1-\beta_t}\mathbf{I}\right]\text{Norm}_{\mathbf{z}_{t-1}}\left[\sqrt{\alpha_{t-1}}\cdot\mathbf{x}, (1-\alpha_{t-1})\mathbf{I}\right] \\
&= \text{Norm}_{\mathbf{z}_{t-1}}\left[\frac{(1-\alpha_{t-1})}{1-\alpha_t}\sqrt{1-\beta_t}\mathbf{z}_t + \frac{\sqrt{\alpha_{t-1}}\beta_t}{1-\alpha_t}\mathbf{x}, \frac{\beta_t(1-\alpha_{t-1})}{1-\alpha_t}\mathbf{I}\right].
\end{aligned}
$$

- $q(\mathbf{z}_t|\mathbf{z}_{t-1}, \mathbf{x}) = q(\mathbf{z}_t|\mathbf{z}_{t-1})$ due to conditional independence

Conditional Diffusion Distribution $q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x})$

From Line 3 to 4, we have:

$$\text{Norm}_{\mathbf{z}_t}\left[\sqrt{1-\beta_t}\cdot\mathbf{z}_{t-1}, \beta_t\mathbf{I}\right] \propto \text{Norm}_{\mathbf{z}_{t-1}}\left[\frac{1}{\sqrt{1-\beta_t}}\mathbf{z}_t, \frac{\beta_t}{1-\beta_t}\mathbf{I}\right].$$

Proof:

We use the Gaussian change of variables identity:

$$\text{Norm}_{\mathbf{v}}\left[\mathbf{A}\mathbf{w}, \mathbf{B}\right] \propto \text{Norm}_{\mathbf{w}}\left[\left(\mathbf{A}^T\mathbf{B}^{-1}\mathbf{A}\right)^{-1}\mathbf{A}^T\mathbf{B}^{-1}\mathbf{v}, \left(\mathbf{A}^T\mathbf{B}^{-1}\mathbf{A}\right)^{-1}\right].$$

■

Conditional Diffusion Distribution $q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x})$

From Line 4 to 5, we have:

$$\text{Norm}_{\mathbf{z}_t}\left[\sqrt{1-\beta_t}\cdot\mathbf{z}_{t-1}, \beta_t\mathbf{I}\right]\text{Norm}_{\mathbf{z}_{t-1}}\left[\sqrt{\alpha_{t-1}}\cdot\mathbf{x}, (1-\alpha_{t-1})\mathbf{I}\right]$$
$$= \text{Norm}_{\mathbf{z}_{t-1}}\left[\frac{(1-\alpha_{t-1})}{1-\alpha_t}\sqrt{1-\beta_t}\mathbf{z}_t + \frac{\sqrt{\alpha_{t-1}}\beta_t}{1-\alpha_t}\mathbf{x}, \frac{\beta_t(1-\alpha_{t-1})}{1-\alpha_t}\mathbf{I}\right].$$

Proof:

We then use the following Gaussian identity to combine the two Gaussians:

$$\text{Norm}_{\mathbf{w}}[\mathbf{a}, \mathbf{A}]\cdot\text{Norm}_{\mathbf{w}}[\mathbf{b}, \mathbf{B}] \quad \propto$$
$$\text{Norm}_{\mathbf{w}}\left[\left(\mathbf{A}^{-1}+\mathbf{B}^{-1}\right)^{-1}\left(\mathbf{A}^{-1}\mathbf{a}+\mathbf{B}^{-1}\mathbf{b}\right), \left(\mathbf{A}^{-1}+\mathbf{B}^{-1}\right)^{-1}\right]$$

■

Simplifying the ELBO

- We can then further express the ELBO into:

$$\begin{aligned}
&\mathrm{ELBO}[\phi_{1\ldots T}] \\
&\quad = \int q(\mathbf{z}_{1\ldots T}|\mathbf{x}) \log \left[\frac{Pr(\mathbf{x}, \mathbf{z}_{1\ldots T}|\phi_{1\ldots T})}{q(\mathbf{z}_{1\ldots T}|\mathbf{x})} \right] d\mathbf{z}_{1\ldots T} \\
&\quad \approx \int q(\mathbf{z}_{1\ldots T}|\mathbf{x}) \left(\log\left[Pr(\mathbf{x}|\mathbf{z}_1, \phi_1)\right] + \sum_{t=2}^{T} \log \left[\frac{Pr(\mathbf{z}_{t-1}|\mathbf{z}_t, \phi_t)}{q(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{x})} \right] \right) d\mathbf{z}_{1\ldots T} \\
&\quad = \mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\left[\log\left[Pr(\mathbf{x}|\mathbf{z}_1, \phi_1)\right]\right] - \sum_{t=2}^{T} \mathbb{E}_{q(\mathbf{z}_t|\mathbf{x})}\left[\mathrm{D}_{KL}\left[q(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{x}) \middle|\middle| Pr(\mathbf{z}_{t-1}|\mathbf{z}_t, \phi_t) \right] \right].
\end{aligned}$$

Proof:

$$= \int q(z_{1\ldots T} \mid x) \log p(x \mid z_1, \phi_1)\, dz_{1\ldots T} + \int q(z_{1\ldots T} \mid x) \sum_{t=2}^{T} \log \left[\frac{p(z_{t-1} \mid z_t, \phi_t)}{q(z_{t-1} \mid z_t, x)} \right] d_{z_{1\ldots T}}$$

$$= \int q(z_1 \mid x) \log p(x \mid z_1, \phi_1)\, dz_1 + \sum_{t=2}^{T} \int q(z_{t-1}, z_t \mid x) \log \left[\frac{p(z_{t-1} \mid z_t, \phi_t)}{q(z_{t-1} \mid z_t, x)} \right] d_{z_t} d_{z_{t-1}}$$

product rule

$$q(z_{t-1}, z_t \mid x) = q(z_{t-1} \mid z_t, x) q(z_t \mid x)$$

■

Simplifying the ELBO

- Finally, we get:

$$\text{ELBO}\left[\phi_{1\ldots T}\right] = \mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\left[\log\left[Pr(\mathbf{x}|\mathbf{z}_1,\phi_1)\right]\right] - \sum_{t=2}^{T}\mathbb{E}_{q(\mathbf{z}_t|\mathbf{x})}\left[\mathrm{D}_{KL}\left[q(\mathbf{z}_{t-1}|\mathbf{z}_t,\mathbf{x})\middle|\middle|Pr(\mathbf{z}_{t-1}|\mathbf{z}_t,\phi_t)\right]\right].$$

where

Conditional Diffusion:

$$q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x}) = \text{Norm}_{\mathbf{z}_{t-1}}\left[\frac{(1-\alpha_{t-1})}{1-\alpha_t}\sqrt{1-\beta_t}\mathbf{z}_t + \frac{\sqrt{\alpha_{t-1}}\beta_t}{1-\alpha_t}\mathbf{x}, \frac{\beta_t(1-\alpha_{t-1})}{1-\alpha_t}\mathbf{I}\right].$$

Diffusion Kernel:

$$q(\mathbf{z}_t|\mathbf{x}) = \text{Norm}_{\mathbf{z}_t}\left[\sqrt{\alpha_t}\cdot\mathbf{x}, (1-\alpha_t)\mathbf{I}\right].$$

Decoder:

$$\begin{aligned} Pr(\mathbf{z}_{t-1}|\mathbf{z}_t,\phi_t) &= \text{Norm}_{\mathbf{z}_{t-1}}\left[\mathbf{f}_t[\mathbf{z}_t,\phi_t],\sigma_t^2\mathbf{I}\right] \\ Pr(\mathbf{x}|\mathbf{z}_1,\phi_1) &= \text{Norm}_{\mathbf{x}}\left[\mathbf{f}_1[\mathbf{z}_1,\phi_1],\sigma_1^2\mathbf{I}\right], \end{aligned}$$

Diffusion Loss Function

- Putting the expressions for $q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x})$, $\Pr(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \phi_t)$, $\Pr(\mathbf{x} \mid \mathbf{z}_1, \phi_1)$ and $q(\mathbf{z}_t \mid \mathbf{x})$, and rearranging, we get the diffusion loss function:

$$L[\phi] = \sum_{i=1}^{I} \Bigg(\overbrace{-\log\Big[\mathrm{Norm}_{\mathbf{x}_i}\big[\mathbf{f}_1[\mathbf{z}_{i1}, \phi_1], \sigma_1^2\mathbf{I}\big]\Big]}^{\text{reconstruction term}} + \sum_{t=2}^{T} \frac{1}{2\sigma_t^2} \Bigg\| \underbrace{\frac{(1-\alpha_{t-1})}{1-\alpha_t}\sqrt{1-\beta_t}\mathbf{z}_{it} + \frac{\sqrt{\alpha_{t-1}}\beta_t}{1-\alpha_t}\mathbf{x}_i}_{\text{target, mean of } q(\mathbf{z}_{t-1}|\mathbf{z}_t,\mathbf{x})} - \underbrace{\mathbf{f}_t[\mathbf{z}_{it}, \phi_t]}_{\text{predicted } \mathbf{z}_{t-1}} \Bigg\|^2 \Bigg),$$

- where $\mathbf{x}_i$ is the ith data point, and $\mathbf{z}_{it}$ is the associated latent variable at diffusion step t.

Reparameterization of Loss Function

- The original diffusion update was given by:

$$\mathbf{z}_t = \sqrt{\alpha_t} \cdot \mathbf{x} + \sqrt{1-\alpha_t} \cdot \boldsymbol{\epsilon},$$

- The data term $\mathbf{x}$ can be expressed as the diffused image minus the noise that was added to it:

$$\mathbf{x} = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{z}_t - \frac{\sqrt{1-\alpha_t}}{\sqrt{\alpha_t}} \cdot \boldsymbol{\epsilon}.$$

- Putting into the loss and rearranging, we get:

$$\begin{aligned} L[\boldsymbol{\phi}] &= \sum_{i=1}^{I} -\log\Big[\text{Norm}_{\mathbf{x}_i}\left[\mathbf{f}_1[\mathbf{z}_{i1}, \boldsymbol{\phi}_1], \sigma_1^2\mathbf{I}\right]\Big] \\ &\quad + \sum_{t=2}^{T} \frac{1}{2\sigma_t^2} \left\| \left(\frac{1}{\sqrt{1-\beta_t}}\mathbf{z}_{it} - \frac{\beta_t}{\sqrt{1-\alpha_t}\sqrt{1-\beta_t}}\boldsymbol{\epsilon} \right) - \mathbf{f}_t[\mathbf{z}_{it}, \boldsymbol{\phi}_t] \right\|^2. \end{aligned}$$

Reparameterization of Network

- We replace the model $\hat{\mathbf{z}}_{t-1} = f_t[\mathbf{z}_t, \phi_t]$ with a new model $\hat{\epsilon} = g_t[\mathbf{z}_t, \phi_t]$, which predicts the noise ϵ that was mixed with $\mathbf{x}$ to create $\mathbf{z}_t$:

$$\mathbf{f}_t[\mathbf{z}_t, \phi_t] = \frac{1}{\sqrt{1-\beta_t}}\mathbf{z}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}\sqrt{1-\beta_t}}\mathbf{g}_t[\mathbf{z}_t, \phi_t].$$

predicts the noise ϵ

- Substituting the new model into the loss produces the criterion:

$$L[\phi] = \sum_{i=1}^{I} -\log\Big[\text{Norm}_{\mathbf{x}_i}\left[\mathbf{f}_1[\mathbf{z}_{i1}, \phi_1], \sigma_1^2\mathbf{I}\right]\Big] + \sum_{t=2}^{T} \frac{\beta_t^2}{(1-\alpha_t)(1-\beta_t)2\sigma_t^2}\Big\|\mathbf{g}_t[\mathbf{z}_{it}, \phi_t] - \epsilon_{it}\Big\|^2$$

$$= \sum_{i=1}^{I} \frac{1}{2\sigma_1^2}\Big\|\mathbf{x}_i - \mathbf{f}_1[\mathbf{z}_{i1}, \phi_1]\Big\|^2 + \sum_{t=2}^{T} \frac{\beta_t^2}{(1-\alpha_t)(1-\beta_t)2\sigma_t^2}\Big\|\mathbf{g}_t[\mathbf{z}_{it}, \phi_t] - \epsilon_{it}\Big\|^2 + C_i$$

Reparameterization of Network

- Substituting in the definitions of $\mathbf{x}$ and $f_1[\mathbf{z}_1, \boldsymbol{\phi}_1]$, the first term simplifies to:

$$\frac{1}{2\sigma_1^2}\left\|\mathbf{x}_i - \mathbf{f}_1[\mathbf{z}_{i1}, \boldsymbol{\phi}_1]\right\|^2 = \frac{1}{2\sigma_1^2}\left\|\frac{\beta_1}{\sqrt{1-\alpha_1}\sqrt{1-\beta_1}}\mathbf{g}_1[\mathbf{z}_{i1}, \boldsymbol{\phi}_1] - \frac{\beta_1}{\sqrt{1-\alpha_1}\sqrt{1-\beta_1}}\boldsymbol{\epsilon}_{i1}\right\|^2.$$

where

$$\mathbf{x} = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{z}_t - \frac{\sqrt{1-\alpha_t}}{\sqrt{\alpha_t}} \cdot \boldsymbol{\epsilon}.$$

$f_1[\mathbf{z}_1, \boldsymbol{\phi}_1]$ can be computed here: $\mathbf{f}_t[\mathbf{z}_t, \boldsymbol{\phi}_t] = \frac{1}{\sqrt{1-\beta_t}}\mathbf{z}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}\sqrt{1-\beta_t}}\mathbf{g}_t[\mathbf{z}_t, \boldsymbol{\phi}_t].$

Reparameterization of Network

- Adding this back to the final loss function yields:

$$L[\boldsymbol{\phi}] = \sum_{i=1}^{I}\sum_{t=1}^{T} \frac{\beta_t^2}{(1-\alpha_t)(1-\beta_t)2\sigma_t^2} \left\| \mathbf{g}_t[\mathbf{z}_{it}, \boldsymbol{\phi}_t] - \boldsymbol{\epsilon}_{it} \right\|^2,$$

- where the additive constants C_i is dropped.

- In practice, the scaling factors are ignored, giving an even simpler formulation:

$$\begin{aligned} L[\boldsymbol{\phi}] &= \sum_{i=1}^{I}\sum_{t=1}^{T} \left\| \mathbf{g}_t[\mathbf{z}_{it}, \boldsymbol{\phi}_t] - \boldsymbol{\epsilon}_{it} \right\|^2 \\ &= \sum_{i=1}^{I}\sum_{t=1}^{T} \left\| \mathbf{g}_t\left[\underbrace{\sqrt{\alpha_t}\cdot\mathbf{x}_i + \sqrt{1-\alpha_t}\cdot\boldsymbol{\epsilon}_{it}}, \boldsymbol{\phi}_t\right] - \boldsymbol{\epsilon}_{it} \right\|^2, \end{aligned}$$

Diffusion Update: $\mathbf{z}_{it}$

Implementation

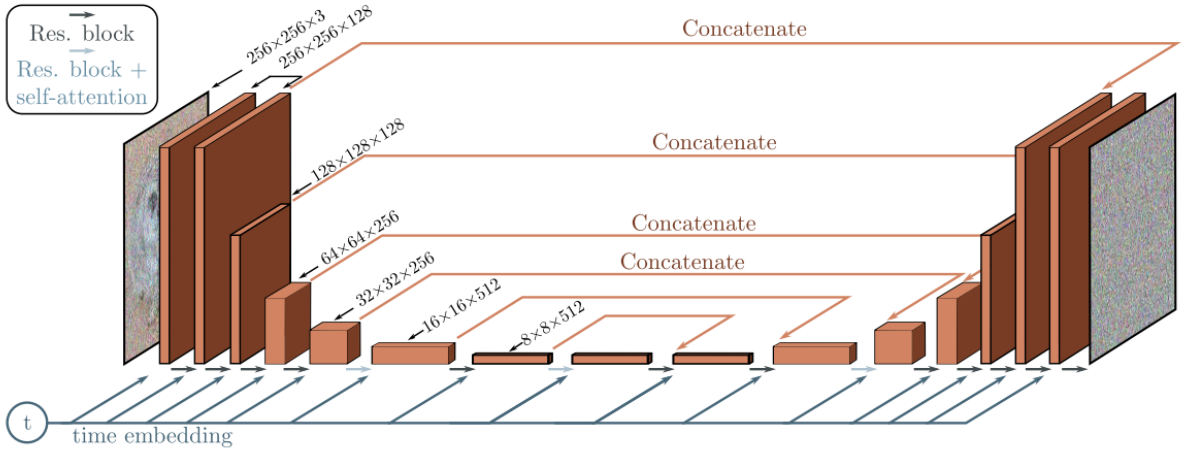
Algorithm 18.1: Diffusion model training

Input: Training data $\mathbf{x}$
Output: Model parameters $\boldsymbol{\phi}_t$
repeat
  for $i \in \mathcal{B}$ **do** // For every training example index in batch
    $t \sim \text{Uniform}[1, \ldots T]$ // Sample random timestep
    $\boldsymbol{\epsilon} \sim \text{Norm}[\mathbf{0}, \mathbf{I}]$ // Sample noise
    $\ell_i = \left\| \mathbf{g}_t\left[\sqrt{\alpha_t}\mathbf{x}_i + \sqrt{1-\alpha_t}\boldsymbol{\epsilon}, \boldsymbol{\phi}_t\right] - \boldsymbol{\epsilon} \right\|^2$ // Compute individual loss
  Accumulate losses for batch and take gradient step
until converged

Algorithm 18.2: Sampling

Input: Model, $\mathbf{g}_t[\bullet, \boldsymbol{\phi}_t]$
Output: Sample, $\mathbf{x}$
$\mathbf{z}_T \sim \text{Norm}_{\mathbf{z}}[\mathbf{0}, \mathbf{I}]$ // Sample last latent variable
for $t = T \ldots 2$ **do**
  $\hat{\mathbf{z}}_{t-1} = \frac{1}{\sqrt{1-\beta_t}}\mathbf{z}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}\sqrt{1-\beta_t}}\mathbf{g}_t[\mathbf{z}_t, \boldsymbol{\phi}_t]$ // Predict previous latent variable
  $\boldsymbol{\epsilon} \sim \text{Norm}_{\boldsymbol{\epsilon}}[\mathbf{0}, \mathbf{I}]$ // Draw new noise vector
  $\mathbf{z}_{t-1} = \hat{\mathbf{z}}_{t-1} + \sigma_t\boldsymbol{\epsilon}$ // Add noise to previous latent variable
$\mathbf{x} = \frac{1}{\sqrt{1-\beta_1}}\mathbf{z}_1 - \frac{\beta_1}{\sqrt{1-\alpha_1}\sqrt{1-\beta_1}}\mathbf{g}_1[\mathbf{z}_1, \boldsymbol{\phi}_1]$ // Generate sample from $\mathbf{z}_1$ without noise

Application to Images



- U-Net as used in diffusion models for images.
- The network aims to predict the noise that was added to the image.
- A single network is used for all time steps, by passing a sinusoidal time embedding through a shallow neural network and adding the result to the channels at every spatial position at every stage of the U-Net.

Conditional Generation

- One approach to conditional synthesis in diffusion models is classifier guidance.
- This modifies the denoising update from $\mathbf{z}_t$ to $\mathbf{z}_{t-1}$ to consider class information c.
- In practice, this means adding an extra term into the final update step in sampling algorithm:

$$\mathbf{z}_{t-1} = \hat{\mathbf{z}}_{t-1} + \sigma_t^2 \frac{\partial \log\left[\Pr(c|\mathbf{z}_t)\right]}{\partial \mathbf{z}_t} + \sigma_t \boldsymbol{\epsilon}.$$

Conditional Generation

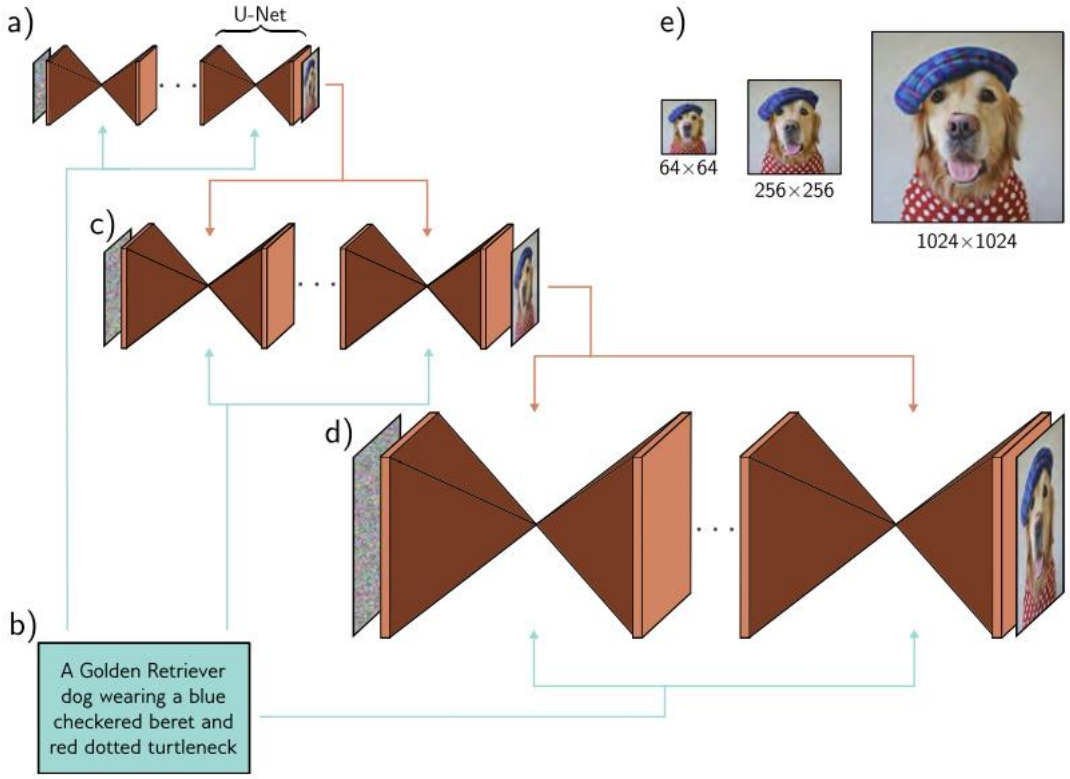
- Classifier-free guidance avoids learning a separate classifier $\Pr(c \mid \mathbf{z}_t)$ but instead in corporates class information into the main model $g_t[\mathbf{z}_t, \phi_t, c]$.

- In practice, this usually takes the form of adding an embedding based on c to the layers of the U-Net in a similar way to how the time step is added.

- This model is jointly trained on conditional and unconditional objectives by randomly dropping the class information during training.

Conditional Generation

- Cascaded conditional generation based on a text prompt.



Summary

- We have looked at how to:

1. Explain the difference between the discriminative and generative models.
2. Describe the concept behind Variational AutoEncoder, and how it can be used to generate new images.
3. Explain the forward (diffusion) and reverse process to noise and denoise an input in the diffusion model.